Cryptocurrency Market Analytics

Final Architecture, Gold Layer, BI and FinOps Report

Students: Emaad Faisal and Qasim Tanveer

Platform: Databricks, PySpark, Delta Lake and Microsoft Power BI

Project scope: Historical cryptocurrency full load with CoinGecko incremental updates

This report summarizes the implemented Bronze, Silver and Gold Lakehouse pipeline, the star-schema analytical model, incremental update strategy, logging, Power BI semantic model, and the main engineering decisions made during the project.

1. Project Overview

The project builds an end-to-end cryptocurrency analytics pipeline. A large historical dataset is used for the initial full load, while CoinGecko market snapshots provide the incremental source. Data is standardized in Bronze and Silver Delta tables and then converted into a dimensional Gold model for Power BI.

Implemented flow

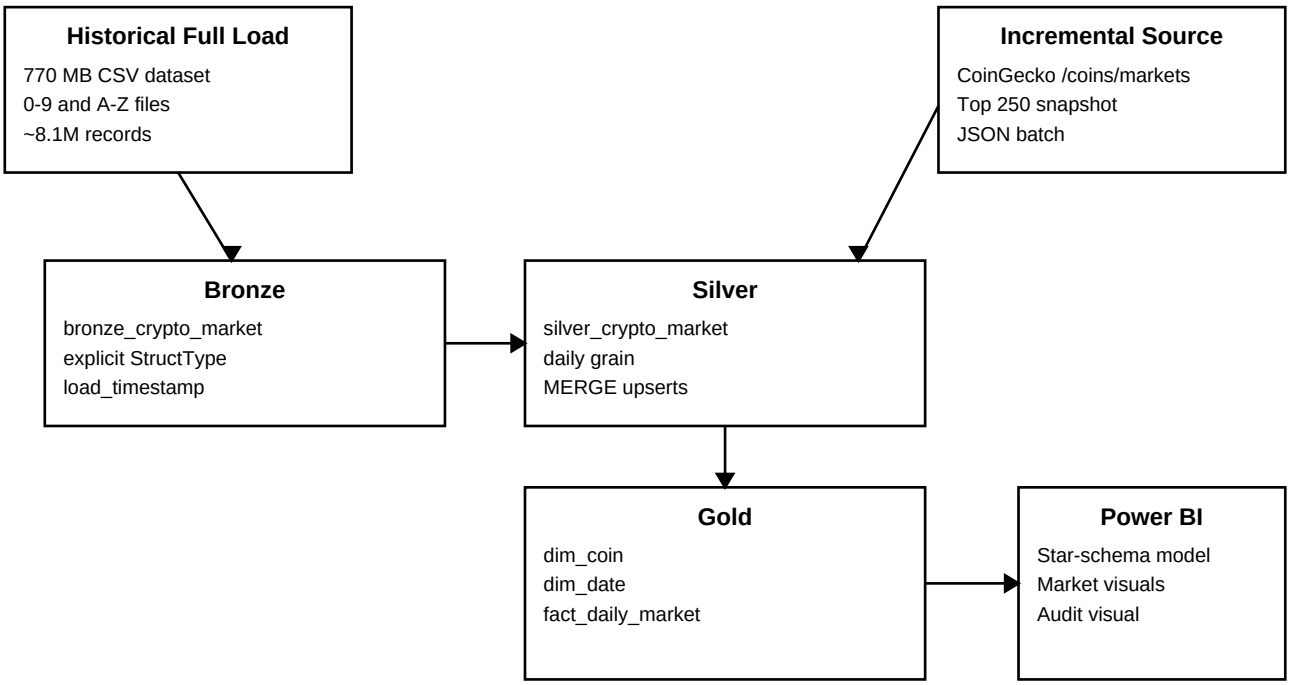


Figure 1. Implemented end-to-end architecture.

Key implementation points

• Historical full load: approximately 770 MB of CSV data, split across files named 0-9 and A-Z, with about 8.1 million market records.

• Incremental load: CoinGecko /coins/markets snapshot for up to 250 coins per batch.

• Primary analytical grain: one cryptocurrency in one quote currency on one observation date.

• Storage: Delta Lake tables for persistent Bronze, Silver, staging, Gold and logging data.

• BI: Power BI Desktop connected directly to Databricks Gold and logging tables.

2. Data Sources and Grain

The historical CSV files contain daily OHLCV-style records with high, open, volume, close, low, Unix timestamp and ticker fields. A ticker such as BTC-USD identifies the cryptocurrency and quote currency. The historical source already behaves as daily data.

The CoinGecko /coins/markets response is a market snapshot rather than a completed daily OHLC candle. The pipeline converts last_updated to a date so multiple snapshots on the same calendar date map to the same daily key. This allows later snapshots on the same day to update the existing row instead of creating duplicates.

Important semantic note: CoinGecko current_price was standardized into the Silver close_price field for schema compatibility. It should be interpreted as the latest snapshot price for the incremental source, not as a mathematically identical completed daily close. high_24h and low_24h are similarly rolling 24-hour measures rather than guaranteed calendar-day highs/lows.

3. Bronze Layer

The Bronze layer preserves source-like historical fields while enforcing an explicit Spark schema. Spark schema inference was not used for the CSV full load. A load_timestamp field records ingestion time.

Column	Type	Purpose
high	DOUBLE	Highest source-period price
open	DOUBLE	Opening source-period price
volume	DOUBLE	Trading volume
close	DOUBLE	Closing source-period price
low	DOUBLE	Lowest source-period price
timestamp	LONG	Unix timestamp
ticker	STRING	Trading pair such as BTC-USD
load_timestamp	TIMESTAMP	Ingestion timestamp

The full-load Bronze table is stored as **bronze_crypto_market**.

4. Silver Layer

The Silver layer converts the technical raw representation into a business-friendly daily model. ticker is split into coin_symbol and quote_currency, timestamp becomes observation_date, price fields are renamed, duplicate coin-date rows are removed, and the table is stored as Delta.

Column	Type	Purpose
coin_symbol	STRING	Standardized uppercase cryptocurrency symbol
quote_currency	STRING	Quote currency such as USD
observation_date	DATE	Daily analytical date
open_price	DOUBLE	Historical opening price; nullable for snapshot incrementals
high_price	DOUBLE	Historical high or mapped 24h high
low_price	DOUBLE	Historical low or mapped 24h low
close_price	DOUBLE	Historical close or mapped current snapshot price
volume	DOUBLE	Historical or CoinGecko total volume
market_cap	DOUBLE	CoinGecko market capitalization; nullable historically
load_timestamp	TIMESTAMP	Processing timestamp

Incremental CoinGecko data is first transformed into the same Silver shape and written to **incremental_silver_market**. A Delta MERGE then upserts it into **silver_crypto_market** using coin_symbol + observation_date. Matched daily keys are updated; unmatched daily keys are inserted. Reprocessing the same batch therefore does not create duplicate daily records.

5. Schema Drift and Data Quality

Schema evolution was used when market_cap was introduced from the CoinGecko source even though the historical CSV files did not provide the field. Historical records receive a nullable DOUBLE market_cap value. The Delta write used mergeSchema=true so the Silver table could evolve without a full pipeline failure.

Incoming CoinGecko measures are explicitly cast to expected data types before the Silver merge. Foreign-key validation is later performed in Gold to verify that every fact row has matching Coin and Date dimension keys.

6. Gold Layer Dimensional Model

The Gold layer uses a star schema rather than copying Silver directly. The fact table keeps the daily measures while descriptive coin and date attributes are separated into dimensions.

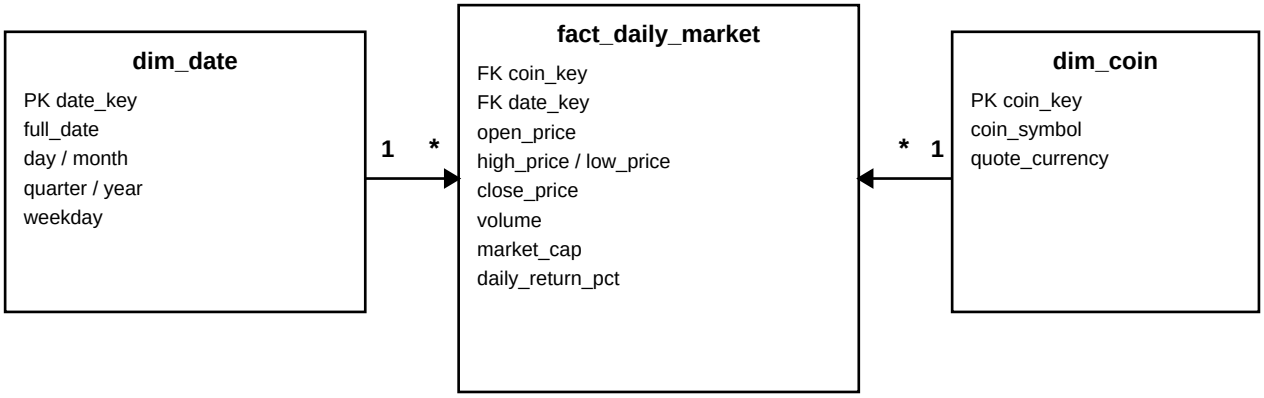


Figure 2. Gold star schema and cardinalities.

Fact table: fact_daily_market

Column	Role
coin_key	FK to dim_coin
date_key	FK to dim_date
open_price	Daily opening price
high_price	High price measure
low_price	Low price measure
close_price	Standardized daily/snapshot price
volume	Trading volume
market_cap	Market capitalization when available
daily_return_pct	Return from previous available price
load_timestamp	Source processing timestamp

Dimension table: dim_coin

Column	Role
coin_key	Primary surrogate-style hash key
coin_symbol	Cryptocurrency symbol
quote_currency	Quote currency

Dimension table: dim_date

Column	Role
date_key	Primary key in YYYYMMDD form
full_date	Calendar date
day / month / year	Calendar attributes
month_name / quarter	Reporting attributes
weekday	Day-of-week attribute

The fact transformation uses a Spark window partitioned by coin_symbol and quote_currency and ordered by observation_date. lag(close_price) obtains the previous available price and daily_return_pct is calculated as ((current - previous) / previous) * 100 when a valid previous price exists.

7. Pipeline Integration and Logging

Gold processing is placed after the Silver incremental MERGE so the Gold tables refresh from the latest silver_crypto_market state. The Gold pipeline creates or refreshes dim_coin, dim_date and fact_daily_market, validates foreign-key relationships, and appends a Silver-to-Gold execution entry to pipeline_execution_logs.

Column	Purpose
run_id	Unique run identifier
layer	Pipeline stage such as Silver-to-Gold
file_processed	Source file or table processed
status	SUCCESS or FAILURE status
rows_processed	Rows handled by the stage
start_time	Execution start timestamp
end_time	Execution end timestamp
execution_time	Timestamp when the log record was written

8. Power BI Semantic Model and Dashboard

Power BI Desktop connects to the Databricks Gold tables and the pipeline_execution_logs table. The semantic model uses the same one-to-many relationships implemented in Gold: dim_coin[coin_key] to fact_daily_market[coin_key], and dim_date[date_key] to fact_daily_market[date_key]. The logging table is intentionally separate from the market star schema.

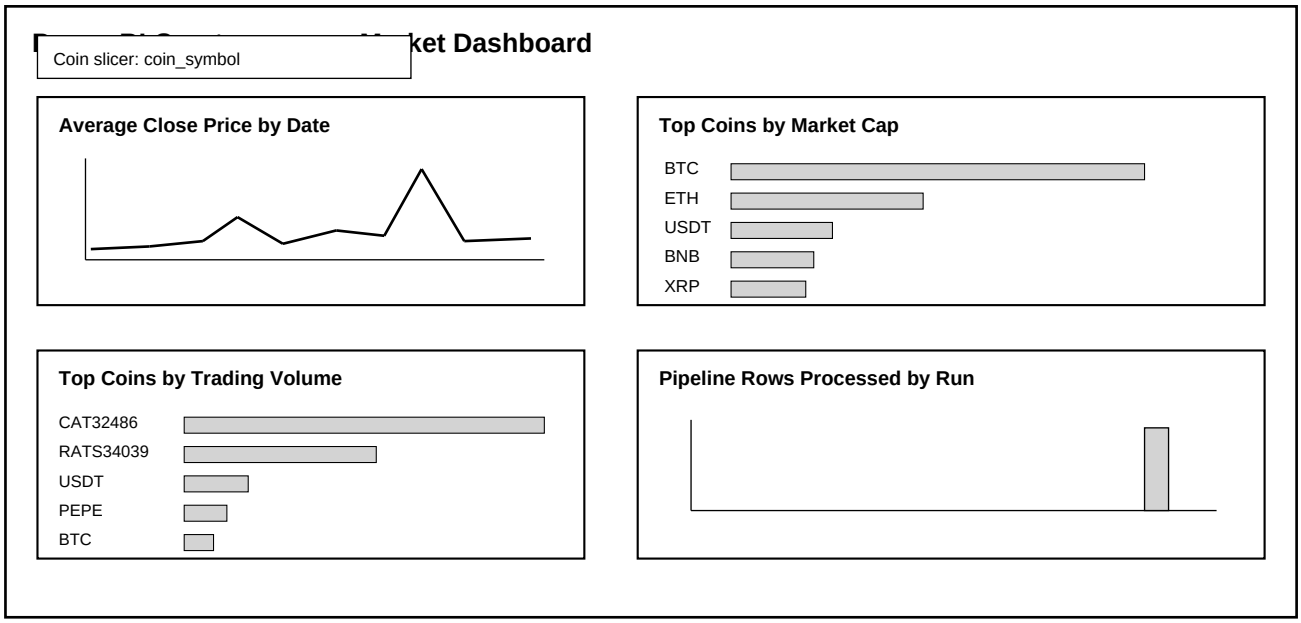


Figure 3. Power BI dashboard visuals implemented for market trends, market capitalization, trading volume and pipeline audit monitoring.

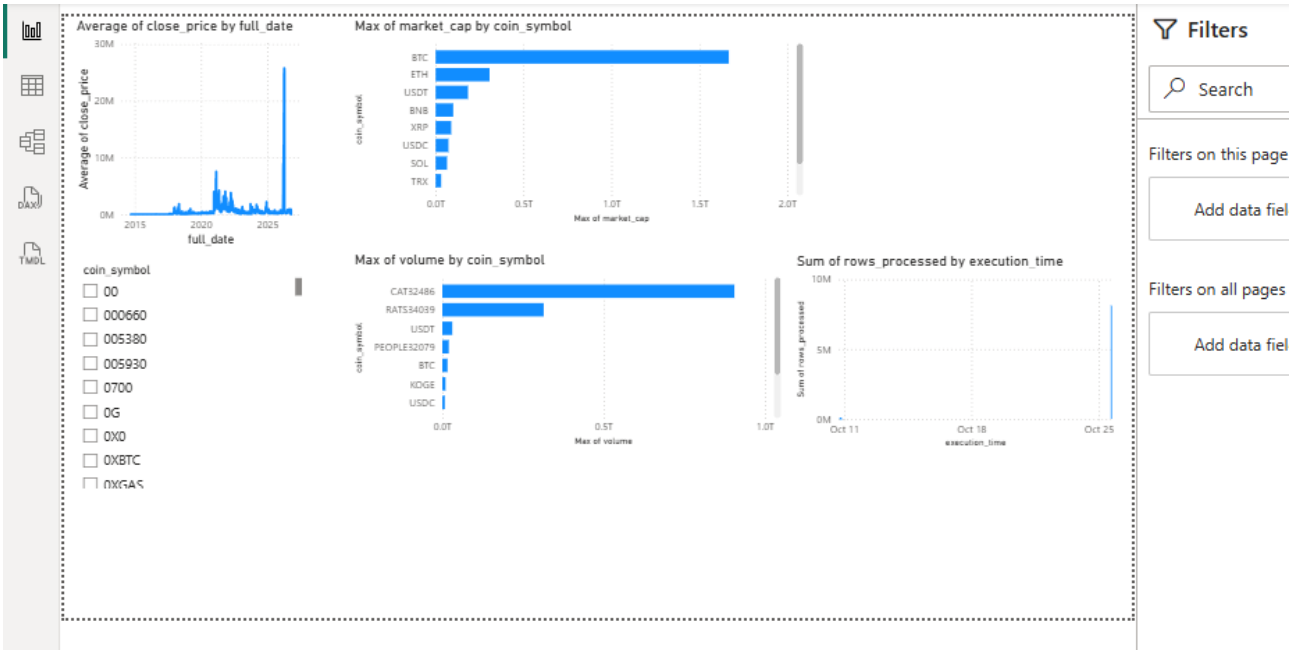


Figure 4. Power BI semantic model showing the Gold star-schema relationships and the separate pipeline logging table.

The completed dashboard contains a price trend over time, Top 10 coins by market capitalization, Top 10 coins by trading volume, a coin slicer, and a pipeline rows-processed chart. The market visuals are driven from the Gold star schema while the audit visual comes from pipeline_execution_logs.

9. Engineering Decision Log

Decision	Reason
Daily grain	Matches historical source and prevents multiple same-day snapshots from becoming duplicate fact rows.
Bronze retained	Preserves source-like data so Silver logic can be re-run without rebuilding source extraction.
Delta Lake	Required for reliable persisted tables, schema evolution and MERGE-based upserts.
MERGE on coin + date	Supports idempotent daily updates and insertion of new observation dates.
market_cap nullable	Historical source does not provide market cap, while CoinGecko does.

Decision	Reason
Gold star schema	Separates descriptive attributes from measures and maps cleanly to Power BI one-to-many relationships.
Deterministic hash coin_key	Provides a stable numeric key from coin symbol and quote currency across refreshes.
Date key YYYYMMDD	Simple integer key that retains natural chronological meaning for BI joins.

Partitioning and optimization

The project did not introduce a separate physical partitioning scheme in the submitted Gold code. The main performance approach was to keep the analytical grain compact, select only required Gold columns, and use Delta tables. OPTIMIZE was considered as an optional Databricks read optimization but was not required for correctness.

PII handling

The selected market datasets contain cryptocurrency market attributes rather than direct personal identifiers. No names, addresses, email addresses, account identifiers or other user-level PII were intentionally ingested into the modeled tables. Therefore no PII masking or tokenization layer was required for the project data used.

10. Spark Execution / DAG Analysis

The most complex Gold calculation is the daily return transformation. It uses a Spark window partitioned by coin_symbol and quote_currency and ordered by observation_date, followed by lag(close_price). Conceptually, Spark must group records by the window partition keys, maintain date order within each partition, evaluate the lag expression, project the return measure, generate Gold keys, and write the resulting Delta fact table.

Because the project notebook ran on Databricks serverless compute, a traditional Spark UI DAG screenshot was not supplied with the material used to generate this report. The notebook can instead expose the formatted physical plan with fact_daily_market_df.explain(mode="formatted") when raw Spark UI visibility is unavailable. If the instructor strictly requires a screenshot, this is the one remaining evidence item that should be inserted before submission.

Expected physical-plan concepts to look for are scan/project operations, partition exchange or repartitioning associated with the window keys, sorting by observation_date within the window, the Window operator for lag(), and the final Delta write. Any claim about a specific shuffle or predicate pushdown should be based on the actual captured plan rather than assumed.

11. FinOps Evaluation

Development was performed with Databricks Free Edition/serverless resources, so a project-specific Azure consumption invoice or paid DBU figure was not captured. For that reason, this report does not invent a monetary cost for the completed runs.

The cost driver for an enterprise deployment would be compute consumed per run multiplied by execution frequency. A production estimate should use the selected Databricks SQL/compute SKU, its current DBU rate, applicable cloud infrastructure charges, and measured incremental-run duration. The annual hourly-run count is 24 x 365 = 8,760 executions, while a once-daily architecture performs 365 executions.

The design already reduces unnecessary cost by using incremental MERGE rather than rebuilding the historical dataset for every new snapshot. Further optimization would include processing only changed dates, limiting selected columns, avoiding unnecessary full-table counts during routine production runs, optimizing the fact table for frequent BI predicates, sizing compute to the incremental workload, and scheduling compute so it is not left running without work.

12. Final Architecture Summary

The final system implements a complete Lakehouse-to-BI path: historical CSV data is loaded through an explicitly typed Bronze layer, standardized in Silver, incrementally refreshed with CoinGecko snapshots using Delta MERGE, transformed into a Gold star schema, logged operationally, and consumed by Power BI through a semantic model that mirrors the Gold relationships.

The design demonstrates full and incremental ingestion, explicit schema handling, idempotent updates, schema evolution, dimensional modeling, BI integration and operational monitoring in one pipeline.

Appendix A - Submission Artifacts

• Final Databricks/PySpark notebook or exported source in the GitHub repository.

• Updated README with Bronze, Silver and Gold data models plus the execution/backfill guide.

• Power BI Desktop .pbix file.

• This final architectural PDF report.

• Spark DAG or formatted physical-plan screenshot should be added if the instructor requires visual DAG evidence.